

claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf_e1b943dd-dac\agent-aaf7b223f7e9ff07b.jsonl`  
- SHA-256: `8a214b3cce4143a0da9eff06a907b107d68caf8e63cdea30cba38563b003b11a`  
- Source modified: `2026-07-06T19:18:59+00:00`  
- Imported at: `2026-07-08T16:00:12+00:00`  
- project: `wf_e1b943dd-dac`  
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`
```

Transcript

1. user (2026-07-06T19:18:19.572Z)

Reviewing GitHub PR #40 of the repo at C:/Users/avidu/Projects/Telegram ? a DRAFT, DOCS-ONLY PR (branch docs/track-f-policy-pipeline).
It adds ONE file: docs/adr/0012-policy-pipeline.md (ADR 0012, status Proposed) ? the Track F design doc for a four-tier "policy pipeline" (Tier 1 metadata gates / Tier 2 topic preferences / Tier 3 content safety / Tier 4 deep inspection), a PolicyStage protocol + StageResult explanation trace, and 7 phased follow-up issues F1-F7.
No code changes. Read the ADR:
git -C C:/Users/avidu/Projects/Telegram show docs/track-f-policy-pipeline:docs/adr/0012-policy-pipeline.md

This is a DESIGN review, not a code review ? there is no gate to run. Judge the ADR as a design contract:
soundness, completeness, internal consistency, accuracy of its claims, and consistency with the codebase and the other ADRs (docs/adr/0001..0011 exist on main; read any you need via 'git -C C:/Users/avidu/Projects/Telegram show main:docs/adr/<file>').
Relevant current code (on main): src/tg_video_filter/models.py (FilterConfig, ContentItem, Source, Policy),
src/tg_video_filter/db.py (load_filter_config/save_filter_config shims, sources.watched gate should_ingest_source, delivery_attempts/record_delivery), src/tg_video_filter/telethon_client.py (the live handler: watched gate -> dedup -> FilterEngine.evaluate -> insert -> deliver). Track E (sources.watched opt-in gate) and Track C (delivery_attempts) just merged.

Ground rules:

- READ actual files, cite ADR section / file:line. You MAY run C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe to check claims (e.g. does FilterConfig ignore/drop unknown JSON keys? does ContentItem carry caption text? is content_item.text populated?).
- This is a DRAFT asking for design feedback ? frame findings as design gaps/risks/inaccuracies, not merge-blockers.
 - Severity: high (a claim that is false, or a design gap that would break an F-phase / cause data loss), medium (unspecified interaction, inconsistency, missing dependency), low (wording, invalid example, minor).
- Report only issues in THIS ADR. No praise. Be concrete: what's wrong, why it matters for implementation.

ADVERSARIALLY VERIFY this design finding ? try to REFUTE it. Read the cited ADR section + any referenced ADR/code,
run C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe checks if useful. CONFIRMED if it genuinely holds against THIS ADR (a real gap/inaccuracy/inconsistency).
REFUTED if the ADR actually addresses it / the claim is wrong / it's out of scope for a design doc. PLAUSIBLE if unsettled. Corrected severity (remember: draft design doc, so calibrate ? false claim/data-loss = high, gap = medium).
FINDING (design): F1 'wire into live-message handler' understates the work: the live path uses the legacy VideoRecord shim, not ContentItem/Source/Policy that PolicyContext requires severity medium | section Implementation phasing ? F1 (Dependencies: None)
summary: PolicyContext is built from ContentItem, Source, Policy, but the live handler operates on VideoMeta/VideoRecord and channel_id/user_id via the deprecated shims; it never materializes a ContentItem, Source, or Policy object in-handler.
why_it_matters: F1 is marked 'Dependencies: None,' but to construct a PolicyContext the handler must first obtain the Source row (it currently uses raw channel_id + should_ingest_source), the inserted ContentItem (it currently gets a VideoRecord back from insert_video), and the account's Policy (it currently reads a FilterConfig via engine_holder). Wiring the pipeline means threading the new domain objects through a hot path that today speaks only the legacy vocabulary. That's real, non-trivial work the 'None' dependency hides, and it may need Track D/command plumbing to select which Policy applies.
evidence: telethon_client.py handler: uses VideoMeta, VideoRecord, db.should_ingest_source(conn,user.id,channel_id), db.insert_video -> returns VideoRecord; engine read from engine_holder (FilterConfig-based). No Source/ContentItem/Policy object is in scope at evaluate time.
rec: Note in F1 that it depends on obtaining Source + inserted ContentItem + resolved Policy in the handler (or on migrating the handler off the VideoRecord shim), and on how the applicable Policy is selected (default vs per-route, routing_rules.policy_id). 'Dependencies: None' is optimistic.

2. assistant (2026-07-06T19:18:22.535Z)

I'll verify this finding by reading the ADR and the relevant code.

3. user (2026-07-06T19:18:24.229Z)

ADR 0012: Policy pipeline ? explainable, tiered, user-customizable content gates

- ****Status****: Proposed
- ****Date****: 2026-07-06
- ****Decision owner****: project owner
- ****Tracking issue****: #31 (Track F of the provider-neutral umbrella, #25)
- ****Depends on****: ADR 0011 (Track B ? domain schema, `policies` table)

Context

The system today evaluates every matched video against a single `FilterConfig` (per account, via `policies.config\_json`). The filter is ****flat****: all rules run unconditionally, the output is a boolean (`matched`), and there is no way to express topic preferences, security policy, or why a specific video passed or failed.

Track F defines a ****policy pipeline**** ? an ordered sequence of stages that run before delivery ? so the system can express richer decisions, explain them, and defer expensive work until after cheap rejections.

What we have today (post-Track-E)

```
| Concept | Current state |
|---|---|
| `FilterConfig` / `Policy` | One JSON blob per account. Flat duration/resolution/size/exclude rules. AND semantics. |
| `FilterEngine` | Evaluates all rules; returns `FilterResult(passed, reasons)`.
```

| `policies` table | One row per account (is_default=1). `config\_json` is the `FilterConfig` blob. |
| Security / topic / deep-inspect | Nothing. Codec/keyword/MIME filters and ffmpeg were deferred to v2 in the original plan. |

What we want

A **policy** should be a pipeline of ordered **stages**, each with a well-defined cost, inputs, and explainable output. The pipeline runs **after** dedup and **before** delivery. Users should be able to express topic preferences ("I like AI and philosophy videos"), security rules ("reject links from sketchy domains"), and view **why** a specific item was accepted or rejected.

Decision

Stage model ? four ordered tiers

The pipeline is split into four tiers, ordered by cost (cheapest first). Later stages only run if all earlier stages pass:

```
...
????????????????????????????????????????????????????????????
?                Policy Pipeline                                ?
?                                                                ?
? Tier 1 ? Metadata gates  (always runs, zero I/O)              ?
?   duration, resolution, size, codec, mime, GIF/round          ?
?   ? extends today's FilterEngine                             ?
?                                                                ?
? Tier 2 ? Topic preferences (opt-in, metadata-only)            ?
?   source allowlist, keyword match, channel topic tags         ?
?   ? user-expressed interests filter the stream                ?
?                                                                ?
? Tier 3 ? Content safety  (opt-in, metadata + light I/O)       ?
?   URL/domain reputation, file metadata sanity,                ?
?   caption text scanning                                       ?
?   ? cheap checks; no download                                 ?
?                                                                ?
? Tier 4 ? Deep inspection  (opt-in, heavy I/O)                 ?
?   ffmpeg, perceptual hash, downloaded content scan           ?
?   ? gated behind explicit user opt-in + budget               ?
????????????????????????????????????????????????????????????
...
```

Stage interface ? `PolicyStage`

Each stage is a pure function (or async function for I/O stages) that takes a `PolicyContext` and returns a `StageResult`:

```
```python
@dataclass
class PolicyContext:
 """Immutable snapshot of what the stage can inspect."""
 content_item: ContentItem # from ADR 0011
 source: Source # the source it came from
 policy: Policy # the owning policy config

@dataclass
class StageResult:
 passed: bool
 reason: str # human-readable explanation
 stage_name: str # e.g. "duration-gate", "url-reputation"
 metadata: dict[str, object] = field(default_factory=dict) # for explainability UI
```

```
class PolicyStage(Protocol):
 """A single gate in the pipeline. Stateless; may be async for I/O stages."""
 async def evaluate(self, ctx: PolicyContext) -> StageResult: ...
 ...
```

The pipeline evaluates stages in order and **short-circuits** on the first failure (don't waste I/O on an already-rejected item). The accumulated `StageResult`` list is the **explanation trace** ? every decision is auditable.

## Topic preferences ? Tier 2

Topic preferences are stored as a JSON array in `policies.config_json`` under a `topics`` key:

```
```json
{
  "min_duration_s": 5,
  "topics": {
    "include": ["AI", "Philosophy", "Soccer"],
    "exclude": ["Politics", "Crypto"],
    "mode": "any" // "any" = match any include topic; "all" = match all
  }
}
```
```

Topic matching is **metadata-only** in v1: source title substring match, message caption keyword match, and (future) channel topic tags from Telegram's channel metadata. No ML classification in the design ? that's a separate issue.

## Security stages ? Tier 3

Security stages are **opt-in** and configured per-policy. The design defines three initial stages (implementation deferred to separate issues):

1. **Source allowlist**: reject items from sources not in the `allowlist`` (an explicit list of trusted `provider_source_ids`).
2. **URL/domain reputation**: check links in captions against a configurable blocklist. Metadata-only ? no page fetch.
3. **Duplicate suppression**: enhanced dedup that compares by `content_fingerprint`` across accounts (already handled by `content_items.dedup_key``; this stage adds cross-policy duplicate detection within an account's delivery streams).

Each security stage records its decision in `StageResult.metadata`` for explainability (e.g. `{"blocked_domain": "sketchy-site.com"}``).

## Deep inspection ? Tier 4

Deep inspection (ffprobe, perceptual hash, downloaded content scan) is **gated behind explicit opt-in** because it violates ADR 0003's no-download principle. The opt-in is per-policy (`deep_inspect: true`` in `config_json``) and the stage has a **budget** (max bytes per item, max concurrent downloads) enforced by the policy engine.

This tier is explicitly deferred to a follow-up issue ? the design only **reserves the slot** in the pipeline model so it doesn't require a schema migration when implemented.

## Policy evaluation flow (post-dedup, pre-delivery)

```
```python
async def evaluate_pipeline(
```

```

    ctx: PolicyContext,
    stages: list[PolicyStage],
) -> tuple[bool, list[StageResult]]:
    results: list[StageResult] = []
    for stage in stages:
        result = await stage.evaluate(ctx)
        results.append(result)
        if not result.passed:
            return False, results # short-circuit
    return True, results
...

```

The caller (telethon_client or future Bot API handler) records the results via `delivery\_attempts.last\_error` (for failed items) or a new `policy\_decisions` table (for explainability ? deferred to implementation).

Schema ? minimal changes

****No schema migration**** is required for this design. The `policies` table already holds a `config\_json` JSON blob. The pipeline is a code concept:

- `policies.config\_json` gains optional `topics`, `allowlist`, `security\_checks`, `deep\_inspect` keys ? all backwards-compatible with the existing `FilterConfig` Pydantic model (extra fields are ignored).
- Future `policy\_decisions` table (deferred) would store the explanation trace for debugging: `(content\_item\_id, policy\_id, stage\_name, passed, reason, metadata\_json, evaluated\_at)`.

Implementation phasing (future issues)

Phase	Scope	Dependencies
---	---	---
F1	Policy engine prototype	`PolicyStage` protocol, `evaluate_pipeline()`, wire into live-message handler None
F2	Topic preferences	`topics` config parsing, source title/caption keyword matching F1
F3	Source allowlist	`allowlist` config, skip non-allowlisted sources F1
F4	URL/domain reputation	Domain blocklist, caption link extraction F1
F5	Deep inspection	ffprobe/perceptual hash, download budget enforcement F1, ADR 0003 waiver
F6	Policy decisions table	Audit trail storage, explainability UI F1
F7	Explainability endpoint	`/why <content_item_id>` command showing the explanation trace F6

Each phase is a separate, mergeable PR ? none are bundled here. This ADR only defines the ****design contract**** the phases code against.

Consequences

****Positive****

- Unifies topic, security, and quality gates into a single explainable framework ? no ad-hoc if-chains scattered across handlers.
- Cheap gates always run first; expensive enrichment is only reached for items that pass the cheap gates.
- `StageResult` list provides an audit trail (explainability) without additional DB lookups at decision time.
- Backwards-compatible: `policies.config\_json` already exists; adding optional keys requires no migration.
- Each implementation phase is independently shippable behind feature flags.

****Negative****

- Four tiers add conceptual complexity vs. today's flat filter.
- `PolicyStage` protocol requires all stages to conform to the same

- interface, which may be awkward for deeply different checks (metadata-only vs. I/O-heavy). Mitigated by the `metadata` dict catch-all.
- Topic matching is string-based substring matching in Phase 1 ? not semantic. A future ML stage would replace it, but it's good enough for a v1.
- Deep inspection violates ADR 0003 and must be explicitly opt-in with a budget to avoid runaway costs.

Alternatives considered

- ****Do nothing; keep flat FilterConfig.**** Rejected ? the umbrella exists to dissolve the conflation of quality, topic, and security into a single boolean. A pipeline is the natural generalization.
- ****Inline each stage in the handler**** (no protocol). Rejected ? makes it impossible to test stages independently, reorder them, or add new ones without touching the hot path.
- ****Use a rules engine / DSL.**** Rejected ? over-engineered for <100 users. A Python list of callables is simpler, testable, and introspectable.
- ****Bundle all phases into a single PR.**** Rejected ? scope discipline. Each phase is independently shippable; this ADR defines the shared interface they code against.

Open questions for the decision owner

1. ****`StageResult.metadata` shape**** ? dict or typed per-stage? Recommend: dict for Phase 1 (flexible), typed in Phase 2 (once the shape stabilizes).
2. ****Policy decisions table**** ? store every decision, or only failures? Recommend: every decision (explainability), with a TTL-based cleanup for old traces.
3. ****Topic matching**** ? substring only, or regex/glob? Recommend: substring (case-insensitive) for v1; natural-language matching in a follow-up.
4. ****Deep inspection budget**** ? per-item byte limit or per-account bandwidth cap? Recommend: per-item limit (10 MB default) with a per-account monthly cap enforced at the app level.

References

- ADR 0003 ? metadata-first probing (the no-download principle that Tier 4 must opt in to violate).
- ADR 0009 ? selective channel scanning (the `topics.allowlist` concept generalizes `sources.watched`).
- ADR 0011 ? provider-neutral domain schema (the `policies` table this design extends, `ContentItem` as the `PolicyContext` input).
- Issue #31 ? Track F (this ADR's tracking issue).
- Issue #25 ? umbrella; this is the final track.

4. assistant (2026-07-06T19:18:26.117Z)

Now let me look at the live handler and the domain objects to verify the claim.

5. user (2026-07-06T19:18:27.146Z)

```

1      """Telethon ingestion: client factory, event handler, and metadata extraction.
2
3      Architecture (see docs/design.md §5):
4
5          NewMessage event
6          -> extract_video_meta(message)    # pure, unit-tested
7          -> dedup via db.get_video()
```

```

8         -> FilterEngine.evaluate(meta)
9         -> db.insert_video(...) + delivery.forward_to_target(...,
target=cfg.delivery_target)
10
11     The metadata extraction is deliberately a **pure** function taking a Telethon
12     ``Message`` and returning a ``VideoMeta | None``. This isolates the only piece
13     of logic that's worth unit-testing (everything else is Telethon/async glue).
14     """
15
16     from __future__ import annotations
17
18     import logging
19     import sqlite3
20     from pathlib import Path
21     from typing import Protocol
22
23     import aiosqlite
24     from telethon import TelegramClient
25     from telethon.events import NewMessage
26     from telethon.tl.types import (
27         Document,
28         DocumentAttributeAnimated,
29         DocumentAttributeVideo,
30         Message,
31         MessageMediaDocument,
32     )
33
34     from tg_video_filter import db
35     from tg_video_filter.commands import ENGINE_KEY
36     from tg_video_filter.delivery import build_link_text, deliver
37     from tg_video_filter.filters import FilterEngine, VideoMeta
38     from tg_video_filter.models import Channel, DeliveryMode, TelegramSession, User,
VideoRecord
39
40     log = logging.getLogger(__name__)
41
42
43     # --- pure metadata extraction -----
44
45
46     def extract_video_meta(message: Message) -> VideoMeta | None:
47         """Extract ``VideoMeta`` from a Telethon ``Message``, or ``None`` if not a video.
48
49         Returns ``None`` when the message has no video/document media. Returns a
50         ``VideoMeta`` (with fields possibly ``None``) for a video whose individual
51         attributes are missing ? the filter engine decides how to treat gaps.
52
53         Telegram stores video metadata in two places:
54         - ``Document``: ``size`` (bytes), ``mime_type`` (str), ``attributes`` (list)
55         - ``DocumentAttributeVideo`` (inside ``attributes``): ``duration`` (float s),
56           ``w`` (width), ``h`` (height)
57
58         Note: we inspect ``message.media`` directly rather than the ``.video``
59         convenience property, because the latter only exists on Telethon's *custom*
60         Message wrapper (applied at runtime) and is ``None`` on the raw TL Message
61         used in tests. A Document is considered a video iff it contains a
62         ``DocumentAttributeVideo``.
63         """
64         media = getattr(message, "media", None)
65
66         # Real Telegram messages wrap documents in MessageMediaDocument.
67         # Unwrap it to get the actual Document.
68         if isinstance(media, MessageMediaDocument):
69             media = media.document
70

```

```

71         if not isinstance(media, Document):
72             return None
73
74         # Only treat this as a video if the document carries a video attribute.
75         has_video_attr = any(isinstance(a, DocumentAttributeVideo) for a in
media.attributes)
76         if not has_video_attr:
77             return None
78
79         meta = VideoMeta(
80             size_bytes=media.size,
81             mime_type=media.mime_type or None,
82         )
83
84         # Find the video attribute for duration/dimensions and round/nosound flags.
85         video_update: dict = {}
86         for attr in media.attributes:
87             if isinstance(attr, DocumentAttributeVideo):
88                 if attr.duration:
89                     video_update["duration_s"] = int(round(attr.duration))
90                 if attr.w:
91                     video_update["width"] = attr.w
92                 if attr.h:
93                     video_update["height"] = attr.h
94                 video_update["is_round"] = attr.round_message # may be None
95                 video_update["nosound"] = attr.nosound # may be None
96                 break
97
98         # Check for GIF (DocumentAttributeAnimated present).
99         is_gif = any(isinstance(a, DocumentAttributeAnimated) for a in media.attributes)
100
101         meta = meta.model_copy(update={**video_update, "is_gif": is_gif})
102         return meta
103
104
105 # --- client factory -----
106
107
108 def build_client(
109     session: TelegramSession, settings: SettingsLike
110 ) -> TelegramClient: # pragma: no cover
111     """Build a Telethon client with a file-backed SQLiteSession (ADR 0010).
112
113     The session path is read from the ``telegram_sessions.session_path`` column
114     (data-driven, no longer derived from the PK at runtime) and resolved
115     against ``settings.sessions_dir``. The directory is created if missing.
116     """
117     sessions_dir = Path(settings.sessions_dir)
118     sessions_dir.mkdir(parents=True, exist_ok=True)
119     # session_path is stored as a basename (e.g. "user_111.session"); join with
120     # the runtime sessions_dir so the migration did not need to know the path.
121     session_path = sessions_dir / session.session_path
122
123     client = TelegramClient(
124         str(session_path),
125         api_id=settings.tg_api_id,
126         api_hash=settings.tg_api_hash,
127     )
128     return client
129
130
131 class SettingsLike(Protocol):
132     """Structural type for the subset of Settings the client factory needs.
133
134     Using a Protocol keeps the module importable in tests without dragging in

```



```

135     the real Settings class. At runtime ``config.Settings`` satisfies this shape.
136     ""
137
138     tg_api_id: int
139     tg_api_hash: str
140     sessions_dir: str
141
142
143     # --- event handler wiring -----
144
145
146     def make_new_message_handler(
147         *,
148         user: User,
149         conn: aiosqlite.Connection,
150         engine_holder: dict[str, FilterEngine],
151         client: TelegramClient,
152         delivery_mode: DeliveryMode = DeliveryMode.FORWARD,
153         delivery_target: str = "me",
154     ) -> _NewMessageCallback:
155         """Return an async callback for ``events.NewMessage``.
156
157         The callback:
158         1. extracts VideoMeta + document_id (skips if not a video),
159         2. dedups via two-tier content dedup (document.id, then metadata fingerprint),
160            plus the existing (channel, message) instance dedup,
161         3. runs the filter engine (read from ``engine_holder`` so config
162            changes via ``/set`` take effect immediately),
163         4. inserts a VideoRecord row,
164         5. delivers a match to ``delivery_target`` (see ADR 0005).
165         """
166
167         async def on_new_message(event: NewMessage.Event) -> None: # pragma: no cover
168             message: Message = event.message # type: ignore[assignment]
169             if message is None:
170                 return # Telethon edge case: event has no message (shouldn't happen)
171             # Only process genuine channel/group posts and private messages we
172             # receive ? outgoing messages (sent by our own account) are ignored.
173             if getattr(message, "out", False):
174                 return
175
176             meta = extract_video_meta(message)
177             if meta is None:
178                 return # not a video
179
180             channel_id = _peer_id(message.peer_id)
181             message_id = message.id
182             document_id = _extract_document_id(message)
183
184             # --- watched gate (ADR 0009, Track E) -----
185             # Opt-in: a source is ingested only when watched=True. An absent
186             # sources row means the channel has never been seen ? treat it as
187             # unwatched and lazily create a watched=0 row for /channels
188             # discoverability (ADR 0009 lines 37-44, 130-131).
189             if not await db.should_ingest_source(conn, user.id, channel_id):
190                 return # skip unwatched / unknown source
191
192             # --- three-tier dedup (cheapest first) ---
193
194             # Tier 0: same (user, channel, message) instance ? already processed.
195             existing = await db.get_video(conn, user.id, channel_id, message_id)
196             if existing is not None:
197                 return
198
199             # Tier 1: same Telegram document.id ? forwarded/cross-posted file.

```

```

200     if document_id is not None:
201         existing = await db.get_video_by_document_id(conn, user.id, document_id)
202         if existing is not None:
203             log.info(
204                 "video_dedup_document_id user=%s doc_id=%s channel=%s msg=%s "
205                 "original_channel=%s original_msg=%s",
206                 user.id,
207                 document_id,
208                 channel_id,
209                 message_id,
210                 existing.channel_id,
211                 existing.message_id,
212             )
213             return
214
215     # Tier 2: metadata fingerprint ? same file re-uploaded (rare but possible).
216     if (
217         meta.size_bytes is not None
218         and meta.duration_s is not None
219         and meta.height is not None
220         and meta.width is not None
221     ):
222         existing = await db.get_video_by_content_fingerprint(
223             conn,
224             user.id,
225             meta.size_bytes,
226             meta.duration_s,
227             meta.height,
228             meta.width,
229         )
230         if existing is not None:
231             log.info(
232                 "video_dedup_fingerprint user=%s size=%s dur=%s h=%s w=%s "
233                 "channel=%s msg=%s original_channel=%s original_msg=%s",
234                 user.id,
235                 meta.size_bytes,
236                 meta.duration_s,
237                 meta.height,
238                 meta.width,
239                 channel_id,
240                 message_id,
241                 existing.channel_id,
242                 existing.message_id,
243             )
244             return
245
246     # Lazily record the channel (best-effort; failures are non-fatal).
247     try:
248         await db.upsert_channel(
249             conn,
250             Channel(user_id=user.id, channel_id=channel_id,
251 title=_channel_title(event)),
252         )
253     except Exception:
254         log.warning(
255             "failed to upsert channel %s for user %s", channel_id, user.id,
256         exc_info=True
257     )
258
259     result = engine_holder[ENGINE_KEY].evaluate(meta)
260     record = VideoRecord(
261         user_id=user.id,
262         channel_id=channel_id,
263         message_id=message_id,
264         document_id=document_id,

```

```

263         duration_s=meta.duration_s,
264         width=meta.width,
265         height=meta.height,
266         size_bytes=meta.size_bytes,
267         mime_type=meta.mime_type,
268         matched=result.matched,
269     )
270     try:
271         inserted = await db.insert_video(conn, record)
272     except sqlite3.IntegrityError:
273         # Lost a dedup race: a concurrent delivery of the same file
274         # (same document_id or same message instance) won the insert.
275         # The three-tier check above already classified this as new, so
276         # the collision means another task is processing (or has
277         # processed) it ? log and skip.
278         log.info(
279             "video_dedup_race user=%s channel=%s msg=%s doc_id=%s",
280             user.id,
281             channel_id,
282             message_id,
283             document_id,
284         )
285         return
286     log.info(
287         "video_seen user=%s channel=%s msg=%s matched=%s reasons=%s",
288         user.id,
289         channel_id,
290         message_id,
291         result.matched,
292         result.reasons,
293     )
294
295     if result.matched:
296         # Build link text for LINK/BOTH modes (None for FORWARD).
297         link_text = None
298         if delivery_mode != DeliveryMode.FORWARD:
299             channel_title = _channel_title(event)
300             link_text = build_link_text(
301                 message_link=getattr(message, "link", None),
302                 duration_s=meta.duration_s,
303                 height=meta.height,
304                 size_bytes=meta.size_bytes,
305                 channel_title=channel_title,
306             )
307         await deliver(
308             client,
309             message,
310             conn,
311             inserted.id,
312             delivery_mode,
313             link_text=link_text,
314             target=delivery_target,
315         )
316
317     return on_new_message
318
319
320 # --- helpers -----
321
322
323 def _extract_document_id(message: Message) -> int | None:
324     """Extract Telegram's ``document.id`` from a video message.
325
326     Returns None if the message has no document media or the id is absent.
327     This is the most reliable content-based dedup key: Telegram reuses the

```

```

328     same ``document.id`` when a file is forwarded across channels.
329     """
330     media = getattr(message, "media", None)
331     if isinstance(media, MessageMediaDocument):
332         media = media.document
333     if isinstance(media, Document):
334         return getattr(media, "id", None)
335     return None
336
337
338 def _peer_id(peer: object) -> int:
339     """Best-effort numeric id for the peer of a message.
340
341     Telegram represents channel/group/user peers as different TL types; we
342     collapse them to a single int for dedup/storage. The exact subclass isn't
343     important ? only that the id is stable per chat.
344     """
345     # Each Peer* type has a field like channel_id / chat_id / user_id.
346     for field in ("channel_id", "chat_id", "user_id"):
347         val = getattr(peer, field, None)
348         if val is not None:
349             return int(val)
350     # Fallback: hash the repr so we never collide with real ids (negative).
351     return -abs(hash(repr(peer)) % (2**31))
352
353
354 def _channel_title(event: NewMessage.Event) -> str | None: # pragma: no cover
355     """Best-effort channel title from the event, if available."""
356     chat = getattr(event, "chat", None)
357     return getattr(chat, "title", None) if chat is not None else None
358
359
360 class _NewMessageCallback:
361     """Structural type annotation for the handler returned by make*_handler."""
362
363     async def __call__(self, event: NewMessage.Event) -> None: ...
364
365
366     __all__ = [
367         "build_client",
368         "extract_video_meta",
369         "make_new_message_handler",
370     ]
371

```

6. user (2026-07-06T19:18:27.399Z)

```

1     """Pydantic data models for users, channels, videos, and filter config.
2
3     These models are the in-memory representation used by the repository layer
4     (`db.py`) and the filter engine (`filters/`). They are deliberately
5     decoupled from Telethon's own types so the filter engine stays pure and
6     unit-testable without a live Telegram connection.
7     """
8
9     from __future__ import annotations
10
11     from datetime import UTC, datetime
12     from enum import Enum
13
14     from pydantic import BaseModel, ConfigDict, Field, field_validator, model_validator
15
16
17     def _utcnow() -> datetime:
18         """Timezone-aware UTC 'now', used as a default factory."""

```

```

19         return datetime.now(UTC)
20
21
22     class DeliveryMode(str, Enum):
23         """How a matched video is delivered to the user.
24
25         - ``FORWARD``: forward the video itself to Saved Messages (re-uploads).
26         - ``LINK``: send a text message with a t.me link + metadata summary.
27         - ``BOTH``: forward the video AND send the link message.
28         """
29
30         FORWARD = "forward"
31         LINK = "link"
32         BOTH = "both"
33
34
35     class Account(BaseModel):
36         """A product identity ? the owner of config, channels, routes, and content.
37
38         Decoupled from any specific provider's session (ADR 0010). ``telegram_id``
39         is the bridge used by the Bot API front-door (``from.id``) and by the
40         legacy userbot path; it is nullable so future providers that don't have a
41         Telegram id (e.g. WhatsApp) can still own an ``Account``.
42         """
43
44         model_config = ConfigDict(frozen=True)
45
46         id: int = Field(description="Stable surrogate PK; the owner key for all downstream
47         tables")
48         telegram_id: int | None = Field(
49             default=None,
50             description="Telegram user id (bot from.id OR userbot account id). UNIQUE when
51             set.",
52         )
53         provider: str = Field(default="telegram", description="Provider namespace; future:
54         'whatsapp'")
55         display_name: str | None = None
56         created_at: datetime = Field(default_factory=_utcnow)
57
58     class TelegramSession(BaseModel):
59         """An optional MTPProto (Telethon) session linked to an :class:`Account`.
60
61         A bot-only user has an ``Account`` row but no ``TelegramSession`` row;
62         that is what makes them structurally invisible to ``manager.run``
63         (ADR 0010). ``session_path`` is data-driven so the file-location
64         convention is explicit rather than derived from the PK at runtime.
65         """
66
67         model_config = ConfigDict(frozen=True)
68
69         account_id: int = Field(description="FK ? accounts.id; PK (one session per account
70         in v1)")
71         phone: str | None = Field(default=None, description="E.164 phone; used only for
72         bootstrap")
73         session_path: str = Field(description="File-backed Telethon session path
74         (user_<id>.session)")
75         enabled: bool = Field(
76             default=True,
77             description="If False, the manager skips this session (not the account)",
78         )
79         created_at: datetime = Field(default_factory=_utcnow)
80         last_login_at: datetime | None = None

```

```

78     class User(BaseModel):
79         """Deprecated: legacy v1 model conflating identity + MTPROTO session.
80
81         Retained as a backwards-compatible view over the new ``accounts`` +
82         ``telegram_sessions`` tables (ADR 0010). New code should use
83         :class:`Account` + :class:`TelegramSession` directly. The ``db`` layer
84         maps ``User`` ? (``Account``, ``TelegramSession``) so existing callers
85         and tests keep working without a flag day.
86         """
87
88         model_config = ConfigDict(frozen=True)
89
90         id: int = Field(description="Telegram user id (== accounts.telegram_id for legacy
rows)")
91         phone: str | None = Field(default=None, description="E.164 phone; used only for
bootstrap")
92         enabled: bool = Field(default=True, description="If False, the manager skips this
session")
93         created_at: datetime = Field(default_factory=_utcnow)
94
95
96     class Channel(BaseModel):
97         """A channel/group a user is a member of. Populated lazily as messages arrive.
98
99         Deprecated: a provider-neutral view over :class:`Source` (ADR 0011). New
100         code should use :class:`Source` directly; the ``db`` layer maps
101         ``Channel`` ? ``Source`` so existing callers and tests keep working.
102         """
103
104         model_config = ConfigDict(frozen=True)
105
106         id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
107         user_id: int
108         channel_id: int = Field(description="Telegram channel/chat id")
109         title: str | None = None
110         seen_first_at: datetime = Field(default_factory=_utcnow)
111
112
113     # --- provider-neutral domain models (ADR 0011, Track B) -----
114     #
115     # These generalize the legacy v1 ``Channel`` / ``VideoRecord`` / ``FilterConfig``
116     # shapes so the schema no longer leaks Telegram-specific ids into every future
117     # provider's code path. ``provider`` columns + provider-prefixed ``dedup_key``
118     # make cross-provider collisions structurally impossible. See ADR 0011 for the
119     # full rationale and the two decisions (Option 1A: tiered ``dedup_key``;
120     # Option 2A: ``policies`` generalizes ``filter_config``).
121
122
123     class Source(BaseModel):
124         """A configured place we ingest content from (ADR 0011).
125
126         Generalizes the legacy :class:`Channel` (1:1 for Telegram today; future
127         providers add their own source kinds). ``provider_source_id`` is the
128         provider-native id (Telegram ``channel_id``, future WhatsApp group id, ?).
129         """
130
131         model_config = ConfigDict(frozen=True)
132
133         id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
134         account_id: int = Field(description="FK ? accounts.id (owner of the source)")
135         provider: str = Field(default="telegram", description="Provider namespace")
136         provider_source_id: str = Field(
137             description="Provider-native source id (Telegram channel_id, ?)"
138         )
139         title: str | None = None

```

```

140     kind: str = Field(default="channel", description="'channel' | 'group' | 'feed'")
141     watched: bool = Field(
142         default=False,
143         description="ADR 0009 selective scanning: ingest from this source iff True",
144     )
145     seen_first_at: datetime = Field(default_factory=_utcnow)
146     last_seen_at: datetime | None = None
147
148
149     class ContentItem(BaseModel):
150         """A normalized content record discovered from any :class:`Source` (ADR 0011).
151
152         Generalizes the legacy :class:`VideoRecord`. Phase 1 stores only videos
153         (``content_type='video'``); ``text`` / ``link_url`` are landed now but stay
154         NULL until later tracks populate them (avoids a future ``ALTER TABLE``).
155         ``dedup_key`` is the provider-neutral dedup key (Option 1A: tiered lookup
156         wrapping ADR 0006, computed by :func:`tg_video_filter.db.compute_dedup_key`).
157         """
158
159         model_config = ConfigDict(frozen=True)
160
161         id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
162         account_id: int = Field(description="FK ? accounts.id")
163         source_id: int = Field(description="FK ? sources.id")
164         provider: str = Field(default="telegram", description="Provider namespace")
165         provider_message_id: str = Field(
166             description="Provider-native message id (Telegram message_id, ?)"
167         )
168         content_type: str = Field(default="video", description="'video' | 'link' |
169 'article'")
170         text: str | None = Field(default=None, description="Caption / post text (nullable in
171 Phase 1)")
172         link_url: str | None = Field(default=None, description="For link/article posts
173 (Phase 1: NULL)")
174         media_ref: str | None = Field(default=None, description="Provider media reference")
175         document_id: str | None = Field(
176             default=None,
177             description="Telegram document.id (kept for TG-specific Tier-1 lookups)",
178         )
179         duration_s: int | None = None
180         width: int | None = None
181         height: int | None = None
182         size_bytes: int | None = None
183         mime_type: str | None = None
184         dedup_key: str = Field(description="Provider-neutral dedup key (Option 1A); unique
185 per account")
186         matched: bool = False
187         seen_at: datetime = Field(default_factory=_utcnow)
188
189     class Policy(BaseModel):
190         """A named, reusable filtering policy owned by an account (ADR 0011, Option 2A).
191
192         Generalizes the legacy per-account ``filter_config`` row so one account can
193         own multiple policies (enables ADR 0009's per-route filter overrides) and so
194         future enrichment/security stages grow into the same ``config_json`` column.
195         Exactly one policy per account carries ``is_default=True``; the legacy
196         ``load_filter_config`` / ``save_filter_config`` shims read/write that row.
197         """
198
199         model_config = ConfigDict(frozen=True)
200
201         id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
202         account_id: int = Field(description="FK ? accounts.id")
203         name: str = Field(description="Policy name; 'default' for the account default

```

```

policy")
201     config_json: str = Field(
202         description="FilterConfig JSON blob (Phase-1 shape); grows in Phase 2 (Track F)"
203     )
204     is_default: bool = Field(
205         default=False, description="Exactly one per account (UNIQUE partial index)"
206     )
207     created_at: datetime = Field(default_factory=_utcnow)
208     updated_at: datetime = Field(default_factory=_utcnow)
209
210
211 # --- Track C: destinations, routing, delivery state (ADR 0011) -----
212 #
213 # These tables were created empty by Track B (_SCHEMA); Track C populates
214 # destinations (backfilled from filter_config_deprecated.config_json), writes
215 # delivery_attempts (replacing the legacy videos.forwarded boolean), and
216 # defines the models for per-destination delivery state. The source/destination
217 # invariant (a channel is either source or destination, never both) is enforced
218 # at the app level by future selective-scanning logic (Track E, #30).
219
220
221 class Destination(BaseModel):
222     """A delivery target ? where matched content is sent (ADR 0011, Track C).
223
224     Generalizes the legacy ``FilterConfig.delivery_target`` field so one account
225     can have multiple named output channels. Backfilled from the existing
226     per-account ``delivery_target``; the default destination is the target used
227     when no explicit routing rule matches.
228     """
229
230     model_config = ConfigDict(frozen=True)
231
232     id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
233     account_id: int = Field(description="FK ? accounts.id")
234     provider: str = Field(default="telegram", description="Provider namespace")
235     target: str = Field(description="Telethon entity: 'me', '@username', or chat id")
236     kind: str = Field(default="saved", description="'saved' | 'channel' | 'chat'")
237     title: str | None = None
238     is_output: bool = Field(
239         default=False,
240         description="ADR 0009: distinguishes output channels from ingest sources",
241     )
242     created_at: datetime = Field(default_factory=_utcnow)
243
244
245 class RoutingRule(BaseModel):
246     """A rule that maps a source through a policy to a destination (ADR 0009).
247
248     Canonical name: ``routing_rules`` (locked in umbrella issue #25). When
249     ``policy_id`` is NULL, the account's default policy is used. When multiple
250     rules match, the highest ``priority`` wins.
251     """
252
253     model_config = ConfigDict(frozen=True)
254
255     id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
256     account_id: int = Field(description="FK ? accounts.id")
257     source_id: int = Field(description="FK ? sources.id")
258     policy_id: int | None = Field(
259         default=None,
260         description="FK ? policies.id; NULL means use account default policy",
261     )
262     destination_id: int = Field(description="FK ? destinations.id")
263     enabled: bool = Field(default=True)
264     priority: int = Field(default=0, description="Higher wins when multiple rules

```



```

match")
265         created_at: datetime = Field(default_factory=_utcnow)
266
267
268     class DeliveryStatus(str, Enum):
269         """Per-destination delivery state, replacing the legacy binary ``forwarded``."""
270
271         PENDING = "pending"
272         DELIVERED = "delivered"
273         FAILED = "failed"
274         RETRYING = "retrying"
275
276
277     class DeliveryAttempt(BaseModel):
278         """Per-destination delivery record ? idempotent via UNIQUE index.
279
280         Replaces the legacy ``videos.forwarded`` boolean (ADR 0011). One row per
281         ``(content_item_id, destination_id)`` pair. On retry, ``attempt_count`` is
282         incremented and ``status``/``last_error`` are updated.
283         """
284
285         model_config = ConfigDict(frozen=True)
286
287         id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
288         content_item_id: int = Field(description="FK ? content_items.id")
289         destination_id: int = Field(description="FK ? destinations.id")
290         route_id: int | None = Field(default=None, description="FK ? routing_rules.id
291         (nullable)")
292         status: DeliveryStatus = Field(default=DeliveryStatus.PENDING)
293         attempt_count: int = Field(default=0, description="Incremented on each retry")
294         last_error: str | None = None
295         created_at: datetime = Field(default_factory=_utcnow)
296         delivered_at: datetime | None = None
297         next_retry_at: datetime | None = None
298
299     class FilterConfig(BaseModel):
300         """Per-user filter configuration.
301
302         v1 rules: duration bounds, minimum height (resolution), and max file size.
303         Stored in the DB as a JSON blob (``filter_config.config_json``) so adding a
304         new field later is backwards-compatible.
305         """
306
307         # Duration bounds in seconds. None means unbounded on that side.
308         min_duration_s: int | None = Field(default=5, ge=0)
309         max_duration_s: int | None = Field(default=7200, ge=0)
310
311         # Resolution: a video passes if its height >= target_height.
312         # None disables the resolution rule.
313         target_height: int | None = Field(default=1080, ge=1)
314
315         # Max file size in mebibytes. None disables the size rule.
316         max_size_mb: int | None = Field(default=2048, ge=1)
317
318         # Content-type exclusions. GIFs and round videos are excluded by default
319         # (their duration/dimensions are usually incidental to real video filters);
320         # silent videos are off by default since some legit clips have no audio.
321         # Existing users' stored configs are unaffected ? the DB JSON blob overrides
322         # these defaults, so this only changes the behaviour for fresh installs.
323         exclude_gif: bool = Field(
324             default=True,
325             description="If True, exclude Telegram GIFs (DocumentAttributeAnimated)",
326         )
327         exclude_round: bool = Field(

```

```

328         default=True,
329         description="If True, exclude circular video messages (round_message=True)",
330     )
331     exclude_nosound: bool = Field(
332         default=False,
333         description="If True, exclude videos without an audio track (nosound=True)",
334     )
335
336     # How matched videos are delivered: forward the video, send a link, or both.
337     # Stored in the same JSON blob, so adding this field is backwards-compatible
338     # (existing rows without it default to FORWARD).
339     delivery_mode: DeliveryMode = Field(default=DeliveryMode.FORWARD)
340
341     # Where matched videos / link messages are sent. Accepts:
342     #   - "me" (default): Saved Messages ? exact v1.1 behaviour.
343     #   - "@username": a public channel the account is a member of.
344     #   - "-100..." or a bare int-as-string: a numeric Telegram chat id
345     #     (private channels without a username).
346     # The string is passed verbatim to Telethon's forward_messages / send_message.
347     # See ADR 0005 for rationale and access requirements.
348     delivery_target: str = Field(
349         default="me",
350         min_length=1,
351         description="Telethon entity to deliver to: 'me', '@username', or a chat id",
352     )
353
354     @field_validator("delivery_target", mode="before")
355     @classmethod
356     def _coerce_delivery_target(cls, v: object) -> str:
357         """Accept int chat ids (e.g. -1001234567890) and coerce to str.
358
359         Telethon accepts both str and int as an entity, but storing as str keeps
360         the JSON blob uniform and lets '@username' / 'me' / numeric ids share
361         one field type.
362         """
363         if v is None:
364             return "me"
365         if isinstance(v, int):
366             return str(v)
367         return str(v)
368
369     @model_validator(mode="after")
370     def _check_duration_bounds(self) -> FilterConfig:
371         """Ensure min_duration_s <= max_duration_s when both are set."""
372         if (
373             self.min_duration_s is not None
374             and self.max_duration_s is not None
375             and self.min_duration_s > self.max_duration_s
376         ):
377             raise ValueError(
378                 f"min_duration_s ({self.min_duration_s}) must be <= max_duration_s "
379                 f"({self.max_duration_s})"
380             )
381         return self
382
383     class VideoRecord(BaseModel):
384         """A seen video message + its metadata. Drives dedup and future /search.
385
386         Deprecated: a provider-neutral view over :class:`ContentItem` (ADR 0011).
387         New code should use :class:`ContentItem`; the ``db`` layer maps
388         ``VideoRecord`` ? ``ContentItem`` so existing callers and tests keep working.
389         """
390
391         model_config = ConfigDict(frozen=True)

```

```

392
393     id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
394     user_id: int
395     channel_id: int = Field(description="Origin Telegram channel id")
396     message_id: int = Field(description="Telegram message id within the channel")
397     document_id: int | None = Field(
398         default=None,
399         description="Telegram document.id ? reused across forwards; Tier-1 content
dedup",
400     )
401
402     # Metadata extracted from the Telethon message (may be None if absent).
403     duration_s: int | None = None
404     width: int | None = None
405     height: int | None = None
406     size_bytes: int | None = None
407     mime_type: str | None = None
408
409     matched: bool = False
410     forwarded: bool = False
411     seen_at: datetime = Field(default_factory=utcnow)
412
413     @property
414     def dedup_key(self) -> tuple[int, int, int]:
415         """The natural key used for deduplication in the ``videos`` table."""
416         return (self.user_id, self.channel_id, self.message_id)
417
418     @property
419     def content_fingerprint(self) -> tuple[int, int, int, int] | None:
420         """Composite key for Tier-2 content dedup: (size_bytes, duration_s, height,
width).
421
422         Returns None if any field is missing (cannot fingerprint).
423         """
424         if None in (self.size_bytes, self.duration_s, self.height, self.width):
425             return None
426         return (self.size_bytes, self.duration_s, self.height, self.width) # type:
ignore[return-value]
427
428
429     # Default filter config shared by the bootstrap onboarding step.
430     DEFAULT_FILTER_CONFIG = FilterConfig()
431

```

7. assistant (2026-07-06T19:18:33.043Z)

The finding is well-confirmed by the handler code. Let me verify a couple of the specific claims in the finding ? that the handler uses `should\_ingest\_source` with raw `channel\_id`, that `insert\_video` returns a `VideoRecord`, and that the engine is FilterConfig-based via `engine\_holder`. All three are directly visible in the code I just read. Let me quickly confirm `insert\_video`'s return type and the ADR's F1/dependency wording.

8. user (2026-07-06T19:18:34.688Z)

```

1572:async def should_ingest_source(
1573-     conn: aiomysqlite.Connection, account_id: int, channel_id: int
1574-) -> bool:
1575-     """Return True if a channel should be ingested (ADR 0009 opt-in gate).
1576-
1577-     - Absent sources row: create it lazily with ``watched=False`` for
1578-       ``/channels`` discoverability; return False (not yet watched).
1579-
1580-
1601:async def insert_video(conn: aiomysqlite.Connection, video: VideoRecord) -> VideoRecord:
1602-     """Deprecated shim: insert into ``content_items`` (ADR 0011).
1603-

```

```

1604-     Resolves the legacy ``channel_id`` to a :class:`Source` row (upserting it
1605-     if absent), computes the provider-neutral ``dedup_key`` via
1606-     :func:`compute_dedup_key`, and inserts a :class:`ContentItem`. The legacy
1607-     ``VideoRecord`` shape is reconstructed from the inserted row.
--
1698:async def load_filter_config(conn: aiosqlite.Connection, user_id: int) -> FilterConfig:
1699-     """Deprecated shim: load the account's default :class:`Policy` as FilterConfig.
1700-
1701-     ADR 0011 (Option 2A): ``filter_config`` is renamed to
1702-     ``filter_config_deprecated``; this reads ``policies WHERE is_default=1``
1703-     instead. Falls back to :data:`FilterConfig` defaults if no policy exists.
1704-     """
--
1711:async def save_filter_config(conn: aiosqlite.Connection, user_id: int, cfg: FilterConfig)
-> None:
1712-     """Deprecated shim: upsert the account's default :class:`Policy`."""
1713-     existing = await get_default_policy(conn, user_id)
1714-     await upsert_policy(
1715-         conn,
1716-         Policy(
1717-             id=existing.id if existing is not None else None,

```

9. assistant (2026-07-06T19:18:44.072Z)

Confirmed. `insert_video` returns a `VideoRecord` (line 1601), `should_ingest_source` takes raw `channel_id` (line 1573), and the filter config path goes through the `FilterConfig/Policy` shim. The handler operates entirely in the legacy vocabulary (`VideoMeta`, `VideoRecord`, `channel_id`, `engine_holder[ENGINE_KEY]`) and never materializes a `Source`, `ContentItem`, or `Policy` object.

The ADR's F1 row lists "wire into live-message handler" with "Dependencies: None," while the `PolicyContext` dataclass explicitly requires `content_item: ContentItem`, `source: Source`, and `policy: Policy`. The finding is accurate: constructing that context requires threading three domain objects the handler doesn't currently hold, plus resolving which policy applies (default vs `routing_rules.policy_id`). The `db` shims deliberately hide these behind the legacy view, so F1 is genuinely more than "None."

One calibration note for severity: this is a phasing/scoping understatement in a draft design doc, not a false factual claim or a data-loss risk. The design is implementable? the objects all exist and the shims can be bypassed? F1 just underestimates the wiring effort and omits the policy-selection dependency. That maps to medium, matching the reported severity.

10. user (2026-07-06T19:18:59.227Z)

Structured output provided successfully